

1 Implementation Plan

1.1 Strategy

This project follows a two-level strategy:

Top-Down Architecture Freeze

+

Bottom-Up Module Implementation

The full Linux-capable multicore RISC-V architecture is fixed first. After that, modules are implemented one by one in dependency order.

The target CPU is:

- RV64GC
- Dual-core
- Out-of-order execution
- Linux-capable
- Sv39 virtual memory
- M/S/U privilege modes
- L1/L2 cache hierarchy
- MESI cache coherence
- CLINT and PLIC interrupt support

RISC-V defines RV64I as the 64-bit base integer ISA, while extensions such as M, A, F, D, C, Zicsr, and Zifencei are layered on top of the base ISA. The privileged architecture defines Machine, Supervisor, and User privilege levels, where Unix-like operating systems require M, S, and U modes.

1.2 Phase 0: Architecture Freeze

1.2.1 Goal

Before writing RTL, freeze all major architectural decisions.

1.2.2 Decisions

Item	Decision
ISA	RV64GC
Base ISA	RV64I
Extensions	M, A, F, D, C, Zicsr, Zifencei
Privilege modes	M, S, U
Virtual memory	Sv39
Core count	2
Execution model	Out-of-order
Branch prediction	BTB + BHT/PHT + RAS
Register model	Physical register file + rename
Retirement	Reorder buffer
Memory ordering	RVWMO + FENCE
Atomic support	LR/SC + AMO
Cache coherence	MESI
Boot target	Boot ROM → OpenSBI → Linux

1.2.3 Deliverables

docs/architecture.tex
docs/module_list.tex
docs/interfaces.tex
rtl/include/riscv_types_pkg.sv

1.3 Phase 1: Common Types and Basic Datapath Modules

1.3.1 Goal

Create the basic reusable modules that all later stages depend on.

1.3.2 Implementation Order

1. riscv_types_pkg.sv
2. physical_register_file.sv
3. immediate_generator.sv
4. integer_alu.sv
5. decoder.sv
6. pc_unit.sv

1.3.3 Modules

riscv_types_pkg Defines global constants and enums.

XLEN = 64
ILEN = 32
ARCH_REGS = 32

```
PHYS_REGS = 96
ROB_ENTRIES = 64
RS_ENTRIES = 32
LSQ_ENTRIES = 32
```

physical_register_file This should be written as a parameterized physical register file, not a fixed architectural register file.

```
NUM_PREG
NUM_RPORT
NUM_WPORT
XLEN
```

This allows Phase 1 to use it as a simple register file, while Phase 10 reuses it for out-of-order execution.

immediate_generator Supports:

- I-type
- S-type
- B-type
- U-type
- J-type

RISC-V keeps rs1, rs2, and rd in fixed bit positions across formats, while immediate bits are rearranged and sign-extended. [:contentReference\[oaicite:2\]index=2](#)

integer_alu Supports:

```
ADD, SUB, AND, OR, XOR,
SLL, SRL, SRA,
SLT, SLTU
```

decoder Decodes:

```
opcode
rd
rs1
rs2
funct3
funct7
immediate type
instruction class
```

pc_unit Supports:

PC + 4
branch redirect
JAL target
JALR target
trap redirect

1.3.4 Exit Criteria

- Each module has a unit testbench
- All basic RV64I arithmetic and immediate tests pass
- Decoder correctly identifies core RV64I instructions

1.4 Phase 2: Minimal RV64I Single-Core Core

1.4.1 Goal

Connect the Phase 1 modules into a minimal single-core RV64I processor.

1.4.2 Modules

1. instruction_memory.sv
2. data_memory.sv
3. load_store_unit.sv
4. branch_unit.sv
5. simple_core_top.sv

1.4.3 Supported Instructions

- Arithmetic: ADD, SUB, ADDI
- Logic: AND, OR, XOR, ANDI, ORI, XORI
- Shift: SLL, SRL, SRA, SLLI, SRLI, SRAI
- Compare: SLT, SLTU, SLTI, SLTIU
- Load: LB, LH, LW, LD, LBU, LHU, LWU
- Store: SB, SH, SW, SD
- Branch: BEQ, BNE, BLT, BGE, BLTU, BGEU
- Jump: JAL, JALR
- Upper immediate: LUI, AUIPC

1.4.4 Exit Criteria

The core can run small bare-metal programs such as:

```
addi x1, x0, 10
addi x2, x0, 20
add  x3, x1, x2
```

Expected result:

```
x3 = 30
```

1.5 Phase 3: Five-Stage Pipeline

1.5.1 Goal

Upgrade the simple core into an in-order five-stage pipeline.

IF -> ID -> EX -> MEM -> WB

1.5.2 Modules

1. if_id_reg.sv
2. id_ex_reg.sv
3. ex_mem_reg.sv
4. mem_wb_reg.sv
5. forwarding_unit.sv
6. hazard_detection_unit.sv
7. flush_unit.sv

1.5.3 Exit Criteria

The pipeline correctly handles:

- ALU-to-ALU forwarding
- load-use stalls
- branch flushes
- jump redirects

1.6 Phase 4: Machine Mode, CSR, and Trap Support

1.6.1 Goal

Add the minimum privileged architecture needed for firmware.

The privileged manual defines CSRs such as mstatus, misa, mtvec, mepc, mcause, mie, mip, medeleg, mideleg, and mhartid. [:contentReference\[oaicite:3\]index=3](#)

1.6.2 Modules

1. csr_unit.sv
2. privilege_controller.sv
3. trap_controller.sv
4. exception_handler.sv

1.6.3 CSRs

- mstatus
- misa
- mtvec
- mepc
- mcause
- mtval
- mie
- mip
- medeleg
- mideleg
- mhartid

1.6.4 Instructions

- CSRRW
- CSRRS
- CSRRC
- CSRRWI
- CSRRSI
- CSRRCI
- ECALL
- EBREAK
- MRET
- WFI

1.6.5 Exit Criteria

The core can:

- trap to mtvec
- write mepc and mcause
- return with mret
- expose mhartid

1.7 Phase 5: Supervisor Mode

1.7.1 Goal

Add Supervisor mode so Linux can run above OpenSBI.

The privileged manual lists supervisor CSRs including sstatus, stvec, sie, sip, sepc, scause, stval, and satp. [:contentReference\[oaicite:4\]index=4](#)

1.7.2 Modules

1. supervisor_csr_support.sv
2. supervisor_trap_controller.sv
3. privilege_return_unit.sv

1.7.3 CSRs

- sstatus
- stvec
- sepc
- scause
- stval
- sie
- sip
- satp
- sscratch

1.7.4 Instructions

- SRET
- SFENCE.VMA

1.7.5 Exit Criteria

The processor supports:

- M-mode
- S-mode
- U-mode
- trap delegation from M-mode to S-mode

1.8 Phase 6: Timer and Interrupt System

1.8.1 Goal

Add basic interrupt support required by Linux scheduling.

1.8.2 Modules

1. clint.sv
2. mtime.sv
3. mtimecmp.sv
4. msip.sv
5. interrupt_router.sv
6. plic.sv

1.8.3 Exit Criteria

The CPU can receive:

- machine timer interrupt
- software interrupt
- external interrupt

1.9 Phase 7: Atomic Extension

1.9.1 Goal

Implement atomic operations required for synchronization.

RISC-V A extension provides atomic instructions for inter-processor synchronization. [:contentReference\[oaicite:5\]index=5](https://www.riscv.org/specifications/atomic/#contentReference[oaicite:5]index=5)

1.9.2 Modules

1. reservation_set.sv
2. atomic_memory_unit.sv
3. memory_lock_unit.sv

1.9.3 Instructions

- LR.D
- SC.D
- AMOSWAP.D
- AMOADD.D
- AMOAND.D
- AMOOR.D
- AMOXOR.D
- AMOMIN.D
- AMOMAX.D
- AMOMINU.D
- AMOMAXU.D

1.9.4 Exit Criteria

The processor can execute lock and spinlock-style synchronization code.

1.10 Phase 8: Sv39 Virtual Memory

1.10.1 Goal

Implement Sv39 virtual memory.

The privileged manual defines satp as the Supervisor Address Translation and Protection register and includes Sv39 page-based virtual memory. :contentReference[oaicite:6]index=6

1.10.2 Modules

1. itlb.sv
2. dtlb.sv
3. page_table_walker.sv
4. instruction_address_translator.sv
5. data_address_translator.sv
6. page_fault_handler.sv
7. sfence_vma_unit.sv

1.10.3 Sv39 Virtual Address

VA[38:30] = VPN[2]

VA[29:21] = VPN[1]

VA[20:12] = VPN[0]

VA[11:0] = page offset

1.10.4 Exit Criteria

The processor can:

- translate virtual addresses
- fill TLB entries
- perform page table walks
- raise instruction page faults
- raise load page faults
- raise store page faults
- flush translation state using SFENCE.VMA

1.11 Phase 9: Linux Bring-Up, Single Core

1.11.1 Goal

Boot real software on one core.

1.11.2 Boot Flow

```
Boot ROM
  |
  v
OpenSBI
  |
  v
Linux Kernel
  |
  v
BusyBox Init
  |
  v
Shell
```

1.11.3 Modules

1. boot_rom.sv
2. uart.sv

3. device_tree_support
4. simple_dram_model.sv

1.11.4 Exit Criteria

The CPU prints Linux boot logs through UART.

1.12 Phase 10: Cache Subsystem

1.12.1 Goal

Replace simple memory access with a realistic cache hierarchy.

1.12.2 Modules

1. l1_i_cache.sv
2. l1_d_cache.sv
3. cache_miss_handler.sv
4. cache_refill_unit.sv
5. shared_l2_cache.sv

1.12.3 Exit Criteria

Linux still boots with L1 caches enabled.

1.13 Phase 11: Out-of-Order Backend

1.13.1 Goal

Upgrade the core from in-order execution to out-of-order execution.

1.13.2 Modules

1. register_alias_table.sv
2. free_list.sv
3. rename_checkpoint.sv
4. reorder_buffer.sv
5. reservation_station.sv
6. issue_logic.sv
7. common_data_bus.sv
8. wakeup_logic.sv
9. rob_commit_logic.sv

1.13.3 Exit Criteria

The processor can:

- rename registers
- dispatch instructions into ROB and RS
- issue ready instructions out of order
- write results through the common data bus
- commit instructions in program order
- recover from branch misprediction
- recover from precise exceptions

1.14 Phase 12: Branch Prediction

1.14.1 Goal

Add performance-oriented frontend prediction.

1.14.2 Modules

1. btb.sv
2. bht.sv
3. pht.sv
4. ras.sv
5. branch_predictor.sv
6. branch_update_unit.sv
7. misprediction_recovery_unit.sv

1.14.3 Exit Criteria

The frontend predicts conditional branches, jumps, and returns, and the backend can recover from wrong-path execution.

1.15 Phase 13: Dual-Core Integration

1.15.1 Goal

Instantiate two harts.

1.15.2 Modules

1. core0
2. core1
3. hart_start_controller.sv
4. memory_arbiter.sv
5. interrupt_router.sv

1.15.3 Exit Criteria

Both cores can:

- boot
- report different mhartid values
- receive separate interrupts
- access shared memory

1.16 Phase 14: MESI Cache Coherence

1.16.1 Goal

Make dual-core cached memory correct.

1.16.2 Modules

1. mesi_controller.sv
2. snoop_logic.sv
3. coherence_directory.sv
4. cache_invalidate_unit.sv
5. cache_writeback_unit.sv

1.16.3 Exit Criteria

Two cores can share memory correctly while running Linux.

1.17 Final Success Criteria

The final CPU is successful if it can:

- execute RV64GC binaries
- boot OpenSBI
- boot Linux

- run BusyBox shell
- support M/S/U privilege modes
- support Sv39 virtual memory
- support timer and external interrupts
- support atomic instructions
- run two harts
- maintain coherent shared memory
- execute instructions out of order